

Relational Database Distillation: From Structured Tables to Condensed Graph Data

Xinyi Gao
The University of Queensland
Brisbane, Australia
xinyi.gao@uq.edu.au

Jingxi Zhang
The University of Queensland
Brisbane, Australia
jingxi.zhang@student.uq.edu.au

Lijian Chen
The University of Queensland
Brisbane, Australia
lijian.chen@uq.edu.au

Tong Chen
The University of Queensland
Brisbane, Australia
tong.chen@uq.edu.au

Lizhen Cui
Shandong University
Jinan, China
clz@sdu.edu.cn

Hongzhi Yin^{*}
The University of Queensland
Brisbane, Australia
h.yin1@uq.edu.au

Abstract

Relational databases (RDBs) underpin the majority of global data management systems, where information is structured into multiple interdependent tables. In social media platforms, for instance, massive user-generated data are organized across related tables such as users, posts, comments, and interactions, enabling large-scale analysis and predictive modeling of social behaviors. To effectively use the knowledge within RDBs for predictive tasks, recent advances leverage graph representation learning to capture complex inter-table relations as multi-hop dependencies. Despite achieving state-of-the-art performance, these methods remain hindered by the prohibitive storage overhead and excessive training time, due to the massive scale of database and the computational burden of intensive message passing across interconnected tables. To alleviate these concerns, we propose and study the problem of Relational Database Distillation (RDD). Specifically, we aim to distill large-scale RDBs into compact heterogeneous graphs while retaining the predictive power (i.e., utility) required for training graph-based models. Multi-modal column information is preserved through node features, and primary-foreign key relations are encoded via heterogeneous edges, thereby maintaining both data fidelity and relational structure. To ensure adaptability across diverse downstream tasks without engaging the traditional, inefficient bi-level distillation framework, we further design a kernel ridge regression-guided objective with pseudo-labels, which produces quality features for the distilled graph. Extensive experiments on multiple real-world RDBs demonstrate that our solution substantially reduces the data size while maintaining competitive performances on classification and regression tasks, creating an effective pathway for scalable learning with RDBs.

CCS Concepts

• Computing methodologies → Neural networks.

^{*}Corresponding author.



This work is licensed under a Creative Commons Attribution 4.0 International License.
WWW '26, Dubai, United Arab Emirates
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2307-0/2026/04
<https://doi.org/10.1145/3774904.3792734>

Keywords

Relational database, Dataset distillation, Multi-tasks, Efficiency.

ACM Reference Format:

Xinyi Gao, Jingxi Zhang, Lijian Chen, Tong Chen, Lizhen Cui, and Hongzhi Yin. 2026. Relational Database Distillation: From Structured Tables to Condensed Graph Data. In *Proceedings of the ACM Web Conference 2026 (WWW '26)*, April 13–17, 2026, Dubai, United Arab Emirates. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3774904.3792734>

1 Introduction

Relational databases (RDBs) are the foundation of global data management systems [25] and estimated to store over 70% of the world's structured data [4]. By organizing information into multiple inter-linked tables connected through primary-foreign key relations, RDBs capture both rich structural dependencies and diverse statistical patterns. This generalizable framework serves as the foundational backbone for data storage across critical domains, including finance, education, and e-commerce [24, 43, 46, 49, 51]. For example, in social media platforms [23, 50], the relational paradigm models users, posts, comments, and interactions as interdependent entities distributed across multiple tables. This enables a wide range of predictive tasks such as user behavior modeling, recommendation and influence estimation, demonstrating the crucial role of RDBs in understanding large-scale systems.

With the rapid growth of data-driven services, RDBs have expanded to unprecedented scales, often containing billions of rows distributed across interdependent tables. While this growth underscores their central role in data-centric applications, it also introduces significant challenges for predictive modeling [37–39], particularly in capturing the heterogeneous table attributes and inter-table dependencies induced by foreign-key relations. In an influence prediction task on social media platforms, for example, modeling user influence requires integrating information across multiple tables. The Users table provides profile and demographic attributes, the Posts and Comments tables contain behavioral and content features, and the Interactions table captures the network structure through user identifiers. Effectively combining these heterogeneous attributes and inter-table dependencies is crucial for accurate influence estimation. However, traditional tabular learning pipelines [8] rely on extensive feature engineering to join and aggregate related table entities. This process is too labor-intensive

to scale and often sacrifices coverage of multi-hop relational information that provides valuable predictive signals.

To alleviate the intricate inter-table dependencies, recent research [2, 3, 6, 29, 35, 53] has introduced a promising paradigm that reformulates relational databases as heterogeneous graphs [20]. In this paradigm, table rows/entities are treated as nodes and primary–foreign key relations are modeled as edges. This reformulation enables heterogeneous graph neural networks (GNNs) [16, 31, 44, 44] to capture inter-table dependencies and multi-hop relations through message passing [18], thereby producing expressive node representations for downstream tasks. Benefiting from graph representation learning, these methods eliminate the need for manual feature engineering and have achieved state-of-the-art performances.

Despite its potential to capture relational information, scaling GNNs to large RDBs remains a formidable challenge. GNNs rely on iterative message passing, which requires repeatedly fetching and aggregating neighboring information across multiple interconnected tables. In large-scale RDBs, this leads to prohibitive training time and high computational cost, compounded by substantial storage overhead. As a result, while GNNs advance predictive performance, their scalability barriers limit practical deployment in resource-constrained or time-sensitive environments [11, 17].

To address these limitations, we investigate Relational Database Distillation (RDD), a new data-centric paradigm that compresses a large relational database into a compact representation while preserving its essential predictive information. The synthetic data enables much more efficient model training yet achieves performance comparable to training on the original large-scale database. While dataset distillation [12, 13, 40] was first explored in computer vision, synthesizing compact data for relational databases introduces several unique challenges due to the nature of tables and relational structures [10, 14]. (i) *Multi-modal attributes within tables*. Columns in relational tables are often heterogeneous in modality [3], ranging from numerical and categorical to temporal values, and they are inherently unordered. Consequently, each modality necessitates dedicated encoding and distillation strategies to preserve the representativeness of the compressed data and ensure its utility for model training and downstream tasks. (ii) *Heterogeneous tables and complex structural dependencies*. RDBs consist of heterogeneous tables linked by foreign-key relations [8], where child tables depend on parent tables. This hierarchical dependency compels conventional generation methods [34] to construct parent tables before child tables, thereby increasing modeling complexity and incurring substantial generation overhead. Preserving such dependencies in a compressed form is particularly challenging, as it requires capturing both the heterogeneity of table attributes and the inter-table relationships. (iii) *Diverse downstream tasks*. Due to their extensive industrial adoption, relational databases are expected to support diverse predictive tasks across a broad range of applications [35]. While dataset distillation has been explored in modalities such as images [1], videos [45] and graphs [15], existing methods are predominantly designed for classification and fail to generalize to other tasks (e.g., regression). Therefore, in various downstream tasks, an effective RDD approach is expected to maintain consistent utility of the synthetic data.

In light of these challenges, we propose Table-to-Graph (T2G), the first relational database distillation framework that compresses large-scale relational databases into compact heterogeneous graphs while preserving predictive utility. T2G addresses the challenge of multi-modal attributes through a clustering-based pretraining objective. It independently encodes each column using lightweight, modality-specific tokenizers, which can be directly applied for table representation and inference during deployment. The clustering process further identifies shared patterns among entities and assigns pseudo-labels that group similar ones, forming the foundation for effective distillation. Subsequently, T2G generates the synthetic heterogeneous graph structure via Stochastic Block Modeling (SBM) [28] guided by pseudo-labels, which explicitly captures inter-cluster relationships and preserves the structural dependencies across tables. This design enables the simultaneous generation of diverse edge types, improving efficiency by avoiding sequential table reconstruction. Finally, to enhance adaptability across tasks, T2G employs a kernel ridge regression (KRR) objective [33] for feature distillation. Guided by both task labels and pseudo-labels, the distillation process enhances data quality and ensures adaptability for both classification and regression tasks.

The main contributions of this paper are three-fold:

- We introduce and formalize relational database distillation (RDD), a novel, practical, yet challenging research problem in large-scale RDB modeling.
- We propose Table-to-Graph (T2G), a novel RDD framework that distills relational databases into compact heterogeneous graphs. T2G employs a clustering-based objective to independently condense multi-modal attributes and leverages SBM to capture heterogeneous dependencies across tables. Furthermore, it generalizes to both classification and regression tasks, ensuring applicability in diverse predictive scenarios.
- Through extensive experiments on real-world relational databases, we show that T2G significantly reduces data size while maintaining competitive performances on classification and regression tasks, demonstrating its effectiveness as a scalable paradigm for learning on relational databases. Our codes are available at: <https://github.com/XYGaoG/T2G>

2 Preliminaries

In this section, we first revisit the fundamental concepts of RDBs. Subsequently, we show how to incorporate the graph representation learning in the prediction of RDBs. Finally, we formally define the problem we studied.

2.1 Relational Database

A relational database contains a collection of tables $\mathcal{T} = \{T_1, \dots, T_n\}$ and a set of links between them $\mathcal{L} \subseteq \mathcal{T} \times \mathcal{T}$. Each table $T \in \mathcal{T}$ is defined as a set of entities/rows $T = \{v_1, \dots, v_{n_T}\}$, where each element $v_i \in T$ represents an entity/row. A link $L = (T_{\text{key}}, T_{\text{pkey}}) \in \mathcal{L}$ exists if a foreign key column in table T_{key} references a primary key column in table T_{pkey} .

Each entity $v \in T$ is represented as a triplet $v = (p_v, \mathcal{K}_v, \mathbf{x}_v)$. Specifically, p_v denotes the **primary key** that uniquely identifies the entity v . $\mathcal{K}_v \subseteq \{p_{v'} : v' \in T' \wedge (T, T') \in \mathcal{L}\}$ represents the set of **foreign keys**, which define the links from entity $v \in T$ to

entities $v' \in T'$, where $p_{v'}$ is the primary key of v' in table T' . An entity can contain one or more foreign keys, depending on the schema design. Finally, $\mathbf{x}_v$ denotes the **multi-modal attributes**, which include numerical and categorical features¹. Formally, $\mathbf{x}_v$ is a tuple of attribute values: $\mathbf{x}_v = (\mathbf{x}_v^{\text{num}}, \mathbf{x}_v^{\text{cat}})$, where $\mathbf{x}_v^{\text{num}} = (x_{v,1}^{\text{num}}, \dots, x_{v,d_{\text{num}}}^{\text{num}}) \in \mathbb{R}^{d_{\text{num}}}$ denotes the numerical attributes, $\mathbf{x}_v^{\text{cat}} = (x_{v,1}^{\text{cat}}, \dots, x_{v,d_{\text{cat}}}^{\text{cat}})$ denotes the categorical attributes and each $x_{v,i}^{\text{cat}} \in \{1, \dots, C_i\}$ denotes the discrete category index of the i -th attribute.

All primary keys, foreign keys, and attributes are represented as table columns, and all entities within the same table share the same column schema.

In predictive applications, one specific table serves as the **target type**, denoted as $T_{\text{tgt}} \in \mathcal{T}$. The entities in T_{tgt} are associated with **task labels** $\mathbf{y}_{T_{\text{tgt}}}$, which provide supervision for downstream learning. The remaining tables $\mathcal{T} \setminus \{T_{\text{tgt}}\}$ act as auxiliary sources of information that provide contextual signals through relational links, enabling joint reasoning over the entire RDB.

2.2 Graph Modeling of RDB

The recent advances [3, 6, 8], including relational deep learning (RDL) [35], represents each relational database as a heterogeneous graph, referred to as the **Relational Entity Graph** (REG). Given a relational database $(\mathcal{T}, \mathcal{L})$, these methods first augment the link set by including its inverse: $\mathcal{L}^{-1} = \{(T_{\text{pkey}}, T_{\text{fkey}}) \mid (T_{\text{fkey}}, T_{\text{pkey}}) \in \mathcal{L}\}$. Accordingly, the schema graph is defined as $(\mathcal{T}, \mathcal{R})$, where the node set is $\mathcal{T}$ and the edge set is $\mathcal{R} = \mathcal{L} \cup \mathcal{L}^{-1}$. This describes the table-level structure of data and indicates type definitions. Based on this schema, the REG is defined as a heterogeneous graph where each row in table is represented by a node. REG is denoted as $G = (\mathcal{V}, \mathcal{E}, \phi, \psi)$, where $\mathcal{V}$ is the node set, $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is the edge set, $\phi : \mathcal{V} \rightarrow \mathcal{T}$ is the node-type mapping function, and $\psi : \mathcal{E} \rightarrow \mathcal{R}$ is the edge-type mapping function. Each node $v \in \mathcal{V}$ is assigned a type $\phi(v) \in \mathcal{T}$, and nodes of the same type correspond to entities in the same table. Similarly, each edge $e \in \mathcal{E}$ is assigned a type $\psi(e) \in \mathcal{R}$ as specified by the schema.

By transforming relational databases into REGs, heterogeneous graph neural networks (GNNs) can be applied to encode entities and generate representations for downstream prediction tasks via message passing. The multi-modal input attributes $\mathbf{x}_v$ are first processed by modality-specific encoders [8], producing embeddings for each column that are then fused into a unified entity representation $\{\mathbf{h}_v^{(0)}\}_{v \in \mathcal{V}}$. Subsequently, a heterogeneous GNN iteratively updates the embeddings according to neighboring entities:

$$\mathbf{h}_v^{(i+1)} = f_{\phi(v)} \left(\mathbf{h}_v^{(i)}, \left\| f_R \left(\left\{ g_R(\mathbf{h}_w^{(i)}) \mid w \in \mathcal{N}_R(v) \right\} \right) \mid \forall R = (T, \phi(v)) \in \mathcal{R} \right\| \right), \quad (1)$$

where $f_{\phi(v)}$, f_R , and g_R are differentiable transformation functions, and $\|\cdot\|$ denotes an aggregation operator such as mean, max, or sum. $\mathcal{N}_R(v) = \{w \in \mathcal{V} \mid (w, v) \in \mathcal{E}, \psi(w, v) = R\}$ represents the R -specific neighborhood of node v .

After multiple propagation layers, the final embeddings integrate information from multi-hop heterogeneous neighbors. The embeddings for the target type T_{tgt} are then fed into task-specific

prediction heads for downstream applications such as classification or regression.

2.3 Problem Formulation

We consider a relational database $\mathcal{D} = (\mathcal{T}, \mathcal{L})$, and its schema graph $(\mathcal{T}, \mathcal{R})$. The corresponding REG is $G = (\mathcal{V}, \mathcal{E}, \phi, \psi)$. The entities in T_{tgt} are associated with task labels $\mathbf{y}_{T_{\text{tgt}}}$.

Relational Database Distillation (RDD) aims to construct a compact synthetic database $\mathcal{D}' = (\mathcal{T}', \mathcal{L}')$ as well as the task label $\mathbf{y}'_{T_{\text{tgt}}}$, such that the total number of entities in $\mathcal{T}'$ is substantially smaller than in $\mathcal{T}$, while preserving the predictive information contained in $\mathcal{D}$.

In this work, we reparameterize $\mathcal{D}'$ as the tuple $\mathcal{D}' = (\mathcal{M}, G')$, where $\mathcal{M} = \{M_1, \dots, M_m\}$ is a set of lightweight column tokenizers and each M_i encodes column attributes into compact embeddings. $G' = (\mathcal{V}', \mathcal{E}', \phi', \psi')$ is the synthetic REG, where $\mathcal{V}'$ is the set of synthetic nodes, $\mathcal{E}' \subseteq \mathcal{V}' \times \mathcal{V}'$ the set of edges, $\phi' : \mathcal{V}' \rightarrow \mathcal{T}'$ assigns each node to a table type, and $\psi' : \mathcal{E}' \rightarrow \mathcal{L}'$ assigns each edge to a relation type. The target type entities/nodes are associated with the task label $\mathbf{y}'_{T_{\text{tgt}}}$. $|\mathcal{V}'| \ll |\mathcal{V}|$ and the schema of the synthetic REG is required to match that of the original database. With this formulation, heterogeneous GNN models trained on G' are expected to achieve performance on downstream tasks comparable to those trained on the original G , while requiring significantly less storage and training time.

In the inference stage, the test REG is first constructed from the RDB, and the pretrained tokenizers $\mathcal{M}$ are employed to independently encode each column. Then, the trained GNN model is applied to perform prediction according to the encoded REG and embeddings.

3 Methodologies

We hereby present Table-to-Graph (T2G), a relational database distillation framework that compresses large-scale relational databases into compact heterogeneous graphs. As shown in Figure 1, T2G begins by encoding each table using lightweight, modality-specific tokenizers pretrained with a clustering-based objective, which captures common patterns across entities and produces pseudo-labels through cluster assignments. Guided by these pseudo-labels, T2G employs SBM to reconstruct heterogeneous graph structures and generate a compact REG that maintains the structural dependencies. Finally, entity features in the synthetic REG are distilled through a KRR objective supervised by both task labels and pseudo-labels, retaining task-relevant information and supporting efficient model training.

3.1 Modality-specific Tokenizers

RDBs contain multiple tables, each comprising diverse multi-modal attributes. For clarity, we illustrate the column encoding process using a single table in this section. Formally, each entity $v \in \mathcal{V}$ is characterized by a set of attributes $\mathbf{x}_v = (\mathbf{x}_v^{\text{num}}, \mathbf{x}_v^{\text{cat}})$, where $\mathbf{x}_v^{\text{num}} = (x_{v,1}^{\text{num}}, \dots, x_{v,d_{\text{num}}}^{\text{num}}) \in \mathbb{R}^{d_{\text{num}}}$ denotes the numerical attributes, $\mathbf{x}_v^{\text{cat}} = (x_{v,1}^{\text{cat}}, \dots, x_{v,d_{\text{cat}}}^{\text{cat}})$ denotes the categorical attributes and each $x_{v,i}^{\text{cat}} \in \{1, \dots, C_i\}$ denotes the discrete category index.

¹We follow previous works [26, 27] to treat the temporal features as the numerical features.

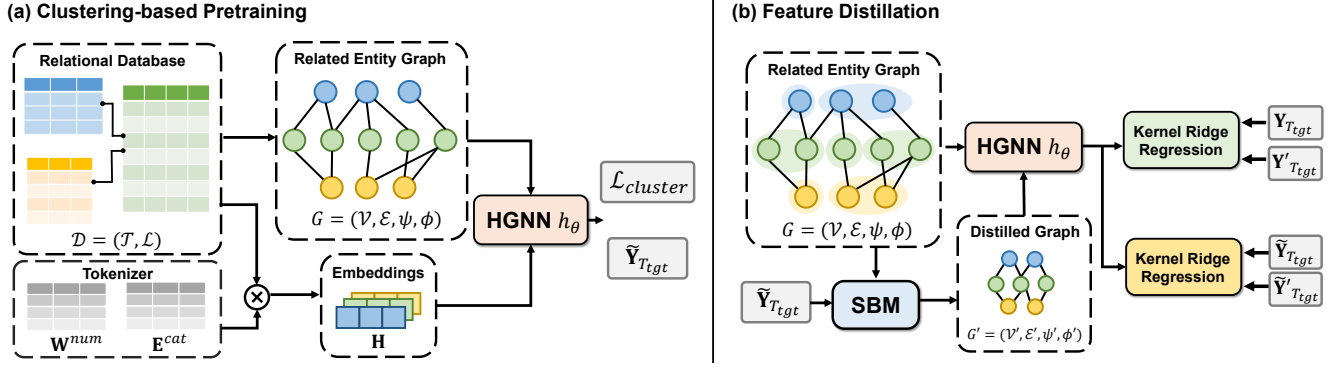


Figure 1: The pretraining and distillation procedures of T2G.

To effectively encode these heterogeneous modalities, T2G employs two types of lightweight, modality-specific tokenizers that independently process each column.

For numerical attributes, each column value $x_{v,i}^{\text{num}}$ is projected into the embedding space using a learnable tokenizer $\mathbf{W}_i^{\text{num}} \in \mathbb{R}^{1 \times d_{\text{token}}}$:

$$\mathbf{h}_{v,i}^{\text{num}} = x_{v,i}^{\text{num}} \mathbf{W}_i^{\text{num}}, \quad i = 1, \dots, d_{\text{num}}. \quad (2)$$

This produces column-wise embeddings $\mathbf{H}_v^{\text{num}} = [\mathbf{h}_{v,1}^{\text{num}}, \dots, \mathbf{h}_{v,d_{\text{num}}}^{\text{num}}] \in \mathbb{R}^{d_{\text{num}} \times d_{\text{token}}}$.

For categorical attributes, each column $x_{v,i}^{\text{cat}}$ takes values from a discrete category set $\{1, \dots, C_i\}$ and is encoded using a learnable lookup table $\mathbf{E}_i^{\text{cat}} \in \mathbb{R}^{C_i \times d_{\text{token}}}$. The embedding for the i -th categorical column is retrieved as:

$$\mathbf{h}_{v,i}^{\text{cat}} = \mathbf{E}_i^{\text{cat}}[x_{v,i}^{\text{cat}}], \quad i = 1, \dots, d_{\text{cat}}, \quad (3)$$

resulting in the embeddings $\mathbf{H}_v^{\text{cat}} = [\mathbf{h}_{v,1}^{\text{cat}}, \dots, \mathbf{h}_{v,d_{\text{cat}}}^{\text{cat}}] \in \mathbb{R}^{d_{\text{cat}} \times d_{\text{token}}}$.

Each attribute is thus represented by a dedicated tokenizer and embedded into a d_{token} -dimensional space. The complete tokenizer parameters consist of $\mathbf{W}^{\text{num}} \in \mathbb{R}^{d_{\text{num}} \times d_{\text{token}}}$ and $\mathbf{E}^{\text{cat}} \in \mathbb{R}^{C \times d_{\text{token}}}$, where C denotes the total number of categories in the table. Notably, $d_{\text{num}} + C$ is proportional to the number of columns in the table and is typically much smaller than the number of entities. Moreover, d_{token} can be set to a small value (i.e., 4 to 8 in practice) while maintaining sufficient representational capacity, ensuring these tokenizers are memory-efficient during deployment.

Importantly, each tokenizer can be viewed as a distilled representation of its corresponding column: the original column values serve as *keys* to retrieve the embeddings from the tokenizer, effectively replacing the raw data with a lightweight yet informative mapping. Thus, storing $\mathcal{M} = \{\mathbf{W}^{\text{num}}, \mathbf{E}^{\text{cat}}\}$ captures the essential column-level knowledge of the database, enabling efficient storage, transfer, and inference while preserving task-relevant information. We empirically justify the compact size of the tokenizers through experiments presented in Section A.2.

To obtain a unified entity representation, the embeddings from both modalities are aggregated through an aggregation function $\text{AGG}(\cdot)$, which can be instantiated as mean, sum, or max pooling:

$$\mathbf{h}_v^{(0)} = \text{AGG}(\mathbf{H}_v^{\text{num}}, \mathbf{H}_v^{\text{cat}}), \quad (4)$$

where $\mathbf{h}_v^{(0)} \in \mathbb{R}^{d_{\text{token}}}$ serves as the initial embedding of entity v .

3.2 Clustering-based Pre-training

After obtaining the embeddings for all entities, the encoded representations of each table are organized into embedding matrices $\mathbf{H}_T = [\mathbf{h}_{v_1}^{(0)}, \dots, \mathbf{h}_{v_{n_T}}^{(0)}] \in \mathbb{R}^{n_T \times d_{\text{token}}}$, where n_T is the number of entities in table $T \in \mathcal{T}$. These embeddings $\mathbf{H} = \{\mathbf{H}_T \mid T \in \mathcal{T}\}$, along with the REG G , are then processed by a heterogeneous graph neural network (HGNN) h_θ as defined in Eq. (1):

$$\mathbf{Z} = h_\theta(G, \mathbf{H}), \quad (5)$$

where $\mathbf{Z} = \{\mathbf{Z}_T \mid T \in \mathcal{T}\}$. The HGNN captures inter-table dependencies by propagating information along primary–foreign key relations and outputs processed embeddings $\mathbf{Z}_T = [\mathbf{z}_{v_1}, \dots, \mathbf{z}_{v_{n_T}}]$ for each table.

To guide representation learning, we employ a clustering-based pre-training objective that encourages embeddings to form semantically coherent groups. For each table T , online clustering is performed on $\mathbf{Z}_T$ to obtain pseudo-labels $\tilde{\mathbf{y}}_T = \{\tilde{y}_{v_1}, \dots, \tilde{y}_{v_{n_T}}\}$, where each $\tilde{y}_{v_i} \in \{1, \dots, n'_T\}$ indicates the cluster assignment of entity v_i . Here, n'_T also corresponds to the number of synthetic entities allocated to table T in the synthetic graph G' , ensuring a one-to-one correspondence between clusters and synthetic entities. The optimization follows a classification objective:

$$\mathcal{L}_{\text{cluster}} = - \sum_{T \in \mathcal{T}} \sum_{v \in T} \log P(\tilde{y}_v \mid \mathbf{z}_v), \quad (6)$$

which jointly updates the encoder parameters and pseudo-label assignments to discover discriminative and well-separated clusters. The resulting pseudo-labels provide the structural foundation for constructing the synthetic REG.

Clustering for target entities. To make the clustering process task-aware, we adopt distinct strategies to generate the pseudo-labels for target type entities, instead of unsupervised clustering:

- For classification tasks, clustering is performed *within each class* of the target entities to avoid semantic mixing between categories. Entities sharing the same task label are clustered independently in the embedding space, ensuring that each cluster represents a coherent subpopulation within its class.
- For regression tasks, clustering is instead performed directly over the continuous labels, allowing the model to capture local label density and provide more fine-grained supervision information.

3.3 Graph Generation via Heterogeneous SBM

Based on the pseudo-labels $\tilde{y}_T$ and clusters identified from the pre-training stage, we next construct the synthetic REG by modeling inter-table dependencies using a heterogeneous Stochastic Block Model (SBM). The pseudo-labels partition table $T \in \mathcal{T}$ into n'_T clusters, each of which corresponds to a synthetic entity in the synthetic graph. Each cluster thus offers a block of prototypes for the corresponding entities in the synthetic REG $G' = (\mathcal{V}', \mathcal{E}', \phi', \psi')$.

To capture the heterogeneous dependencies across tables, we construct a type-specific connectivity matrix $\mathbf{P}_{T_i, T_j} \in [0, 1]^{n'_{T_i} \times n'_{T_j}}$ for each relation $(T_i, T_j) \in \mathcal{R}$. Each entry $\mathbf{P}_{T_i, T_j}(a, b)$ models the empirical probability of linkage between entities belonging to cluster a in T_i and cluster b in T_j , estimated as:

$$\mathbf{P}_{T_i, T_j}(a, b) = \frac{\sum_{(u,v) \in \mathcal{E}_{i,j}} \mathbb{I}[\tilde{y}_u = a, \tilde{y}_v = b]}{\sum_{v \in T_i, w \in T_j} \mathbb{I}[\tilde{y}_v = a, \tilde{y}_w = b]}. \quad (7)$$

Here, $a \in \{1, \dots, n'_{T_i}\}$, $b \in \{1, \dots, n'_{T_j}\}$, and $\mathbb{I}[\cdot]$ denotes the indicator function. Intuitively, $\mathbf{P}_{T_i, T_j}(a, b)$ represents the normalized connection density between the two clusters. Collectively, the set $\{\mathbf{P}_{T_i, T_j}\}_{(T_i, T_j) \in \mathcal{R}}$ defines a heterogeneous SBM that jointly characterizes the inter-table structural dependencies.

Afterwards, we synthesize the relational structure by applying a deterministic thresholding mechanism to control graph sparsity. Specifically, a global sparsity ratio ρ is set as a hyper-parameter, and an adaptive threshold $\tau_{i,j}$ is derived for each relation type to preserve its original density:

$$\tau_{i,j} = \text{Quantile}_{1-\rho}(\mathbf{P}_{T_i, T_j}), \quad (8)$$

where $\text{Quantile}_{1-\rho}$ denotes the $(1-\rho)$ -quantile of all entries in $\mathbf{P}_{T_i, T_j}$. The edges in the adjacency matrix $\mathbf{A}'_{T_i, T_j}$ are then deterministically defined as:

$$\mathbf{A}'_{T_i, T_j}(a, b) = \begin{cases} 1, & \text{if } \mathbf{P}_{T_i, T_j}(a, b) > \tau_{i,j}, \\ 0, & \text{otherwise.} \end{cases} \quad (9)$$

By this means, we can jointly capture relation-specific connectivity patterns across multiple tables. Furthermore, this eliminates the need for hierarchical table generation, enabling parallel feature synthesis across all tables conditioned on the generated graph structure.

3.4 KRR-based Feature Distillation

After obtaining the heterogeneous structure G' , the final stage of RDD focuses on generating features for the synthetic entities. To this end, we employ a HGNN h_θ to encode both the original and synthetic graphs:

$$\mathbf{Z} = h_\theta(G, \mathbf{H}), \quad \mathbf{Z}' = h_\theta(G', \mathbf{H}'), \quad (10)$$

where $\mathbf{H}' = \{\mathbf{H}'_T \mid T \in \mathcal{T}\}$ denotes the features of synthetic graph and each $\mathbf{H}'_T \in \mathbb{R}^{n'_T \times d_{\text{token}}}$ is initialized randomly with the same dimension as $\mathbf{H}_T \in \mathbb{R}^{n_T \times d_{\text{token}}}$.

Our goal is to optimize the synthetic embeddings $\mathbf{H}'$ such that a model trained on the synthetic data achieves comparable predictive performance to the one trained on original data. However, this process inherently involves a nested optimization problem [52], where both the predictive model and the synthetic data must be optimized

recurrently. In single-modality data distillation, such bi-level optimization design has already exposed significant computational complexity and training instability [40, 52]. Thus, straightforwardly adopting bi-level optimization for RDD will further impair the distillation quality, given RDBs' larger scale, multi-modality, and intricate high-order interactions.

To alleviate this issue, we replace the conventional trainable predictor with a *Kernel Ridge Regression* (KRR) model, which provides a closed-form solution for the optimal weights. This enables direct optimization of the synthetic features $\mathbf{H}'$ without iterative model training. Specifically, the KRR prediction model on the synthetic graph is defined as:

$$\mathcal{L}_{\text{KRR}} = \|\mathbf{Y}'_{T_{\text{tgt}}} - \mathbf{Z}'_{T_{\text{tgt}}} \mathbf{W}\|^2 + \lambda \|\mathbf{W}\|^2, \quad (11)$$

where λ is the ridge regularization coefficient, and $\mathbf{W}$ is the learnable weight matrix. $\mathbf{Y}'_{T_{\text{tgt}}}$ denotes matrix-form task labels for target entities. The optimal solution admits a closed form:

$$\mathbf{W}^* = \mathbf{Z}'_{T_{\text{tgt}}}{}^\top \left(\mathbf{Z}'_{T_{\text{tgt}}} \mathbf{Z}'_{T_{\text{tgt}}}{}^\top + \lambda \mathbf{I} \right)^{-1} \mathbf{Y}'_{T_{\text{tgt}}}. \quad (12)$$

To ensure the synthetic data yield high predictive performance on the original data, we minimize the discrepancy between the predictions of the original data and their task labels:

$$\mathcal{L}_{\text{task}}(\mathbf{H}', \mathbf{Y}'_{T_{\text{tgt}}}) = \mathbb{E}_{\theta \sim \Theta} \left[\|\mathbf{Y}_{T_{\text{tgt}}} - \mathbf{Z}_{T_{\text{tgt}}} \mathbf{W}^*\|^2 \right], \quad (13)$$

where the expectation form over parameter θ encourages generalization across different model initializations Θ while KRR guarantees an optimal solution under random feature mappings [32]. KRR-based objective accommodates both regression and classification tasks, where categorical labels are converted into one-hot format for computation.

Extension with pseudo-labels. To further enhance structural consistency and representation alignment, we introduce an auxiliary KRR loss based on the pseudo-labels $\tilde{\mathbf{Y}}_{T_{\text{tgt}}}$ obtained from the clustering-based pretraining:

$$\mathcal{L}_{\text{pseudo}}(\mathbf{H}', \tilde{\mathbf{Y}}'_{T_{\text{tgt}}}) = \mathbb{E}_{\theta \sim \Theta} \left[\|\tilde{\mathbf{Y}}_{T_{\text{tgt}}} - \mathbf{Z}_{T_{\text{tgt}}} \mathbf{W}^*_{\text{pseudo}}\|^2 \right], \quad (14)$$

where

$$\mathbf{W}^*_{\text{pseudo}} = \mathbf{Z}'_{T_{\text{tgt}}}{}^\top \left(\mathbf{Z}'_{T_{\text{tgt}}} \mathbf{Z}'_{T_{\text{tgt}}}{}^\top + \lambda \mathbf{I} \right)^{-1} \tilde{\mathbf{Y}}'_{T_{\text{tgt}}}, \quad (15)$$

and $\tilde{\mathbf{Y}}'_{T_{\text{tgt}}}$ denotes the identity matrix serving as the pseudo-labels for synthetic data.

The overall optimization objective combines task supervision and pseudo-label guidance:

$$\mathcal{L}_{\text{total}}(\mathbf{H}', \mathbf{Y}'_{T_{\text{tgt}}}, \tilde{\mathbf{Y}}'_{T_{\text{tgt}}}) = \mathcal{L}_{\text{task}}(\mathbf{H}', \mathbf{Y}'_{T_{\text{tgt}}}) + \beta \mathcal{L}_{\text{pseudo}}(\mathbf{H}', \tilde{\mathbf{Y}}'_{T_{\text{tgt}}}), \quad (16)$$

where β balances the contribution of the pseudo-label loss. The optimal features and labels are derived from:

$$\mathbf{H}'^*, \mathbf{Y}'_{T_{\text{tgt}}}^* = \arg \min_{\mathbf{H}', \mathbf{Y}'_{T_{\text{tgt}}}} \mathcal{L}_{\text{total}}. \quad (17)$$

This dual-supervision design ensures that the synthetic features and labels encode both task-specific semantics and relational structure, leading to a compact yet expressive synthetic data suitable for diverse downstream applications.

Algorithm 1: The Procedure of Table-to-Graph (T2G)

```

1 Input: Relational database  $\mathcal{D} = (\mathcal{T}, \mathcal{L})$ ; task labels  $Y_{T_{\text{tgt}}}$ .
2 Output: Synthetic graph  $G'$ ; synthetic features  $H'$ ; synthetic labels  $Y'_{T_{\text{tgt}}}$ ; tokenizers  $W^{\text{num}}, E^{\text{cat}}$ .
3  $\triangleright$  Stage 1: Clustering-based Pretraining
4 Initialize  $W^{\text{num}}$  and  $E^{\text{cat}}$ .
5 for  $iter = 1, \dots, K_{\text{pre}}$  do
6   Encode  $T \in \mathcal{T}$  into  $H_T$  by  $W^{\text{num}}$  and  $E^{\text{cat}}$ .
7    $Z = h_{\theta}(G, H)$ .
8   Compute  $\mathcal{L}_{\text{cluster}}$  with Eq. (6).
9   Update  $h_{\theta}, W^{\text{num}}$  and  $E^{\text{cat}}$ .
10 Compute  $\tilde{Y}_T$  for  $T \in \mathcal{T}$ .
11  $\triangleright$  Stage 2: Graph Structure Distillation
12 for  $(T_i, T_j) \in \mathcal{R}$  do
13   Compute  $P_{T_i, T_j}$  using  $\tilde{Y}_{T_i}$  and  $\tilde{Y}_{T_j}$ .
14   Compute  $A'_{T_i, T_j}$  with sparsity  $\rho$ .
15 Compute  $G' = (V', \mathcal{E}', \phi', \psi')$ .
16  $\triangleright$  Stage 3: KRR-based Feature Distillation
17 Initialize  $H'$  and  $Y'_{T_{\text{tgt}}}$ .
18 for  $iter = 1, \dots, K_{\text{distill}}$  do
19   Sample  $\theta \sim \Theta$  and initialize  $h_{\theta}$ .
20    $Z = h_{\theta}(G, H)$ ;  $Z' = h_{\theta}(G', H')$ .
21   Compute KRR model  $W^*$  and  $W^*_{\text{pseudo}}$  with Eq. (12) and Eq. (15).
22   Compute  $\mathcal{L}_{\text{total}}$  with Eq. (16).
23   Update  $H'$  and  $Y'_{T_{\text{tgt}}}$ .

```

3.5 Further Detailed Analysis

Algorithm. The detailed process of T2G is presented in Algorithm 1. Initially, we pre-train the tokenizers and generate pseudo-labels for each entity (lines 4–10). These pseudo-labels are then used to guide the construction of the SBM, which in turn generates the heterogeneous graph for the synthetic graph. In lines 17–23, the KRR-based objective is computed to optimize the synthetic features and labels, incorporating both task labels and pseudo-labels of original data.

Model training and deployment. During downstream model training, the synthetic graph structure G' , features H' , and labels $Y'_{T_{\text{tgt}}}$ are used for efficient model optimization. At inference time, the lightweight tokenizers $\mathcal{M}$ are first applied to obtain column and entity embeddings, after which the trained model is used to perform the prediction task.

The detailed time complexity is provided in the Appendix A.1.

4 Experiments

We design comprehensive experiments to validate the efficacy of our proposed methods and explore the following research questions: **Q1:** Compared to the other dataset reduction and distillation methods, can T2G achieve better performance? **Q2:** Can the synthetic data generated by T2G generalize well to different model architectures? **Q3:** How do the different components, i.e., the SBM module, loss terms, and pseudo-labels, influence the performance of T2G? **Q4:** How do the hyperparameters (the weight β and the sparsity ρ)

Table 1: Summary of Databases. Reg. and Cla. denote the regression and classification tasks, respectively.

Database	#Rows	#Columns	#Tables	Train/Val/Test	Task
Rossmann	1,015,159	16	2	637,954/177,620/201,815	Reg.
Walmart	310,707	17	3	154,759/77,679/78,359	Reg.
Airbnb	1,020,723	20	2	608,911/207,429/204,383	Cla.
IMDB	2,377,044	19	6	1,133,141/623,680/620,223	Cla.

affect the behavior of T2G? **Q5:** How effectively does T2G reduce storage size compared to the original database?

The results for Q5 are deferred to Appendix A.2.

4.1 Experimental Setup

Databases & Baselines. We evaluate our proposed methods on 4 real-world databases: Rossmann, Walmart, Airbnb and IMDB, using the same split method as in the RDBs benchmark [26]. These databases differ in the types of relationships, as well as in the number of tables and columns. A summary of key characteristics is provided in Table 1.

We evaluate the proposed T2G against a diverse set of representative dataset distillation and sampling baselines. Specifically, we compare with the following methods: (1) Random: randomly selects a subset of entities. (2) Herding [47]: selects representative entities that are closest to the cluster center. (3) K-Center [7]: chooses entities that maximize coverage in the feature space by minimizing the maximum pairwise distance. (4) Coarsening [21]: constructs a smaller graph by merging similar entities while preserving topological properties. (5) FreeHGC [30]: a training-free dataset distillation method for heterogeneous graphs that selects entities according to influence metrics. (6) HGCond [9]: a dataset distillation method designed for heterogeneous graphs, which matches model gradients between the original and synthetic data.

Since these methods are not originally designed for tabular or RDBs, they do not consider column-wise encoding. To ensure a fair comparison, we apply all baselines on top of the encoded entity embeddings obtained by pretrained tokenizers, which are supervised with task labels.

Implementations. We evaluate three compression ratios ($r = N'/N$) for each database, where N denotes the number of entities in the original *training set* and N' represents the number of synthetic entities. The tokenizer embedding dimension d_{token} and HGNN hidden dimension d are set to 8 and 64, respectively. For HGNN used in both distillation and evaluation stages, we follow prior work [8, 22] by adopting the GraphSAGE [19] architecture and extending it to support heterogeneous REGs. The regression and classification tasks are evaluated by MAE and AUC score, respectively.

Hyper-parameters. The hyper-parameters are determined through a grid search on the validation set. Following [22], the HGNN layer is set as the number of tables in the RDBs. The aggregation function used for entity embedding generation is set to the mean. The loss weight β is tuned from the set $\{0, 0.1, 1, 10, 20\}$, and the sparsity ρ is explored within the range $(0, 1)$. The ridge regularization coefficient λ is set as $1e-2$. The learning rate for the distillation process is determined through a search over the set $\{0.1, 0.01, 0.001\}$. The weight decay is $5e-4$.

Table 2: Comparison of model performance trained with different dataset reduction methods. Values are reported as mean $\pm$ standard deviation. “Whole Database” denotes the model trained on the full original RDB. r is the compression ratio. Rossmann and Walmart are regression tasks evaluated by MAE, while Airbnb and IMDB are classification tasks evaluated by AUC (%).

Database	r	Baselines						Ours	Whole
		Random	Herding	K-Center	Coarsening	FreeHGC	HGCond	T2G	Database
Rossmann ↓	0.02%	381.7±8.3	237.0±11.2	324.5±9.1	250.1±9.1	229.9±5.4	215.8±2.0	194.4±3.0	188.5±0.9
	0.05%	361.4±8.4	232.2±11.2	271.4±7.6	237.8±6.9	218.4±5.9	212.1±4.5	194.8±3.1	
	0.10%	352.9±9.1	221.8±13.2	261.3±9.7	236.0±6.9	219.2±6.6	213.1±1.4	192.8±4.1	
Walmart ↓	0.10%	18736±2598	16793±1223	16967±653	15136±799	15185±640	14012±647	8989±255	8790±272
	0.20%	16454±1303	16142±1054	16223±494	14325±393	14141±719	14145±600	9070±342	
	0.40%	16086±832	15179±991	15492±404	14140±322	14260±883	14072±599	8848±282	
Airbnb ↑	0.02%	53.6±0.6	56.5±0.5	54.9±1.5	57.1±1.6	58.6±0.8	62.8±1.2	65.5±0.5	69.0±0.1
	0.05%	56.2±1.1	57.0±1.3	57.8±2.0	58.9±1.2	59.7±1.1	62.6±0.7	66.1±0.7	
	0.10%	57.5±1.9	58.3±1.2	57.9±2.2	59.0±1.1	60.9±1.9	62.6±0.6	66.6±0.5	
IMDB ↑	0.02%	50.9±0.2	52.0±0.1	52.3±0.1	54.0±0.4	56.4±0.8	60.4±1.2	65.2±1.2	68.5±0.9
	0.05%	51.1±0.3	52.3±0.5	52.0±1.7	55.4±0.5	57.2±2.3	60.3±1.0	64.8±1.7	
	0.10%	53.1±0.9	54.2±0.3	54.8±1.8	55.9±2.4	58.3±0.1	60.8±1.1	65.5±1.3	

Computing Infrastructure. The codes are written in Python 3.9 and Pytorch 1.12.1. The experiments were conducted on a server equipped with Intel(R) Xeon(R) Gold 6326 CPUs at 2.90GHz and NVIDIA GeForce A40 GPUs with 48GB of memory.

4.2 Effectiveness Comparison (Q1)

Table 2 summarizes the results across all databases, compression ratios, and baselines. Our proposed T2G consistently achieves the best performance, demonstrating its strong ability to preserve predictive information while drastically reducing data size.

Traditional selection-based baselines such as Random, Herding, and K-Center exhibit unstable results. In particular, K-Center heavily depends on the number of selected samples and performs poorly under extreme compression. Graph-based distillation methods generally perform better by modeling structural dependencies. Among them, HGCond synthesizes data by matching gradients between the original and synthetic graphs, achieving the strongest performance and confirming the effectiveness of data synthesis. FreeHGC show limited improvement, as it relies on simple influence metrics instead of learnable feature generation.

Across different compression ratios, T2G maintains stable performance with minimal degradation. In addition, the results show that a lower compression ratio does not necessarily lead to worse performance, which is consistent with previous findings in dataset distillation studies [52]. This observation indicates that effective RDD can retain high predictive accuracy even under substantial compression.

In terms of task types, our proposed T2G effectively handles both regression and classification tasks across multiple databases and compression ratios. Overall, these results highlight the effectiveness of T2G in preserving predictive power while achieving substantial data reduction.

Our evaluated databases exhibit diverse characteristics. For example, Airbnb contains highly imbalanced categorical columns, with imbalance ratio ranging from 2% to 0.001%. Despite these skewed distributions, our method addresses imbalance through

Table 3: The generalizability comparison of data reduction methods. Compression ratio $r = 0.1\%$.

Models	SAGE	GIN	EDGE	HGT	HAN	MLP	Avg.
Rossmann ↓							
Random	352.9	318.3	340.7	348.9	317.7	347.6	337.7
Herding	221.8	254.5	265.8	242.8	245.4	241.7	245.3
K-Center	261.3	285.4	296.8	305.3	276.3	300.3	287.6
Coarsening	236.0	255.9	276.5	257.3	296.4	297.4	269.9
FreeHGC	219.2	246.2	254.6	246.8	233.1	246.2	241.0
HGCond	213.1	214.6	207.4	247.1	221.4	240.6	224.0
T2G (Ours)	192.8	196.2	190.9	213.7	216.5	221.3	205.2
Airbnb ↑							
Random	57.5	55.4	55.7	56.1	56.9	54.5	56.0
Herding	58.3	56.8	57.9	57.3	57.4	57.1	57.5
K-Center	57.9	56.1	56.9	56.9	57.2	57.1	57.0
Coarsening	59.0	57.3	57.4	57.5	57.7	56.8	57.6
FreeHGC	60.9	57.9	58.6	58.0	58.2	57.9	58.6
HGCond	62.6	62.7	62.6	59.4	62.5	62.8	62.1
T2G (Ours)	66.6	64.7	64.9	64.4	65.2	63.5	64.9

independent tokenizers that assign each category a separate embedding. These embeddings are independently optimized on the original data, preventing majority categories from overwhelming minority representations.

In addition, the relation sparsity of our evaluated databases ranges from 0.1% to 2%. Despite this variation, our SBM module adaptively generates synthetic edges using the cluster-level sparsity defined in Eqs. (7)–(9). By modeling connection probabilities between clusters rather than relying on global sparsity, it captures local interaction patterns and handles diverse sparsity effectively.

4.3 Generalizability Comparison (Q2)

The synthetic data is expected to generalize across diverse downstream model architectures. To thoroughly evaluate this property, we assess the performance of baseline methods under diverse experimental settings. We first conduct evaluations using a unified heterogeneous graph neural network framework [36], combined

Table 4: Ablation study on three databases. The compression ratio is set to 0.1%. The prediction tasks for Rossmann and Walmart are regression evaluated by MAE, while Airbnb is a classification task evaluated by AUC (%).

Methods	Rossmann ↓	Walmart ↓	Airbnb ↑
w/o SBM	199.9±4.0	9334±218	65.0±0.6
w/o $\mathcal{L}_{pseudo}$	194.5±3.5	9120±224	66.1±0.4
w/o pseudo-labels	207.4±4.3	9536±255	64.9±0.5
Ours	192.8±4.1	8989±255	66.6±0.5

with different message passing paradigms, including GraphSAGE (SAGE) [19], GIN [48], and EdgeCNN (EDGE) [42]. Furthermore, we examine two specialized heterogeneous graph models—HGT [20] (transformer-based) and HAN [41] (attention-based)—to further assess the adaptability of the synthetic data. In addition, we also include a simple MLP as a non-graph model to verify the generalization capability of the synthetic data beyond message passing architectures.

The detailed results are summarized in Table 3. We observe that models trained on the synthetic data produced by T2G consistently achieve comparable performance across all architectures. Among these models, GraphSAGE yields the best performance due to its architectural alignment with the model used during distillation. In addition, the MLP, which discards the graph structure, still achieves comparable performance. This result suggests that the synthetic data effectively preserve the underlying structural information. When compared with other baselines, T2G significantly outperforms competing methods, demonstrating its strong ability to preserve predictive information and generalize effectively across diverse heterogeneous modeling paradigms.

4.4 Ablation Study (Q3)

To evaluate the contribution of each component in T2G, we conduct an ablation study by disabling specific modules. As shown in Table 4, we first compare T2G without the SBM-based graph generation, where the graph structure is instead constructed by selecting the original edges following [9, 30]. Next, we remove the pseudo-label guidance in the feature distillation stage, denoted as “w/o $\mathcal{L}_{pseudo}$ ”. Finally, we entirely remove the pseudo-label in the process, referred to as “w/o pseudo-labels”. In this setting, the pretraining stage is replaced by task-supervised training using task labels only, the SBM module is replaced by edge selection from the original database, and feature distillation relies solely on task labels.

When the SBM-based graph generation is replaced by sampling edges from the original REG, the relational connectivity among entities is not properly preserved, leading to structural information loss and a noticeable performance drop. In contrast, incorporating pseudo-label guidance during feature distillation allows each synthetic feature to be explicitly aligned with a coherent group of original entities. This divide-and-conquer mechanism enhances the representation consistency within each cluster and proves particularly beneficial for regression tasks, where capturing continuous label correlations is essential. Finally, when pseudo-label modeling is entirely removed from all stages of T2G, the performance degrades substantially, underscoring the pivotal role of pseudo-labels in guiding both structure generation and feature alignment.

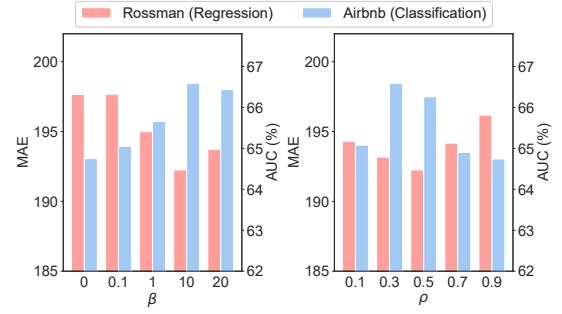


Figure 2: Hyper-parameter analysis on Rossmann and Airbnb. The compression ratios are 0.1%.

4.5 Hyper-parameter Sensitivity Analysis (Q4)

Figure 2 presents the effect of β and ρ on both regression (Rossmann) and classification (Airbnb) tasks, where lower MAE and higher AUC indicate better performance. The hyper-parameter β controls the relative weight of the pseudo-label loss. As shown in the left sub-figure, increasing β initially improves performance on both tasks by encouraging the synthetic features to align more closely with cluster-level semantics captured by the pseudo-labels. However, overly large β values may overemphasize pseudo-labels and weaken overall supervision from task labels, leading to slight degradation. For graph sparsity ρ (right sub-figure), moderate sparsity yields the best performance. A smaller ρ retains essential structural relations and avoids noise, consistent with findings in graph structure learning [54] that sparser graphs often promote generalization. Yet, when ρ becomes too small, insufficient edge information limits message propagation, causing performance to drop.

5 Conclusion

In this paper, we presented Table-to-Graph (T2G), the first framework for relational database distillation, which compresses large-scale relational databases into compact heterogeneous graphs while preserving predictive performance. T2G introduces a clustering-based pretraining objective that enables lightweight, modality-specific tokenization for multi-modal table attributes. Moreover, a SBM mechanism is employed to reconstruct heterogeneous graph structures guided by pseudo-labels. To support both classification and regression tasks, T2G incorporates a KRR-based distillation objective that effectively transfers predictive knowledge into the synthetic features. Extensive experiments on real-world relational databases show that T2G significantly reduces storage requirements while maintaining competitive performance. As an initial attempt at relational database distillation, T2G focuses on a single target table and task, leaving extensions to cross-table and cross-task settings for future work.

Acknowledgments

The Australian Research Council partially supports this work under the streams of Future Fellowship (Grant No. FT210100624), the Discovery Project (Grant No. DP240101108 and DP260100326), and the Linkage Project (Grant No. LP230200892 and LP240200546).

References

- [1] George Cazenavette, Tongzhou Wang, Antonio Torralba, Alexei A Efros, and Jun-Yan Zhu. 2022. Dataset distillation by matching training trajectories. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 4750–4759.
- [2] Tianlang Chen, Charilaos Kanatsoulis, and Jure Leskovec. 2025. Relgnn: Composite message passing for relational deep learning. *arXiv preprint arXiv:2502.06784* (2025).
- [3] Milan Cvitkovic. 2020. Supervised learning on relational databases with graph neural networks. *arXiv preprint arXiv:2002.02046* (2020).
- [4] DB-Engines. 2023. DBMS Popularity Broken Down by Database Model. https://db-engines.com/en/ranking_categories. Accessed: 2023-12-31.
- [5] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The Faiss library. *arXiv preprint arXiv:2401.08281* (2024).
- [6] Vijay Prakash Dwivedi, Sri Jaladi, Yangyi Shen, Federico López, Charilaos I Kanatsoulis, Rishi Puri, Matthias Fey, and Jure Leskovec. 2025. Relational Graph Transformer. *arXiv preprint arXiv:2505.10960* (2025).
- [7] Reza Zanjirani Farahani and Masoud Hekmatfar. 2009. *Facility location: concepts, models, algorithms and case studies*. Springer Science & Business Media.
- [8] Matthias Fey, Weihua Hu, Kexin Huang, Jan Eric Lenssen, Rishabh Ranjan, Joshua Robinson, Rex Ying, Jiaxuan You, and Jure Leskovec. 2024. Position: Relational deep learning-graph representation learning on relational databases. In *Forty-first International Conference on Machine Learning*.
- [9] Jian Gao, Jianshe Wu, and Jingyi Ding. 2024. Heterogeneous Graph Condensation. *IEEE Transactions on Knowledge and Data Engineering* (2024).
- [10] Xinyi Gao, Tong Chen, Yilong Zhang, Wentao Zhang, Quoc Viet Hung Nguyen, Kai Zheng, and Hongzhi Yin. 2024. Graph condensation for inductive node representation learning. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 3056–3069.
- [11] Xinyi Gao, Tong Chen, Wentao Zhang, Yayong Li, Xiangguo Sun, and Hongzhi Yin. 2024. Graph condensation for open-world graph learning. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 851–862.
- [12] Xinyi Gao, Yayong Li, Tong Chen, Guanhua Ye, Wentao Zhang, and Hongzhi Yin. 2025. Contrastive graph condensation: Advancing data versatility through self-supervised learning. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 661–672.
- [13] Xinyi Gao, Guanhua Ye, Tong Chen, Wentao Zhang, Junliang Yu, and Hongzhi Yin. 2025. Rethinking and accelerating graph condensation: A training-free approach with class partition. In *Proceedings of the ACM on Web Conference 2025*. 4359–4373.
- [14] Xinyi Gao, Hongzhi Yin, Tong Chen, Guanhua Ye, Wentao Zhang, and Bin Cui. 2025. Robgc: Towards robust graph condensation. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [15] Xinyi Gao, Junliang Yu, Tong Chen, Guanhua Ye, Wentao Zhang, and Hongzhi Yin. 2025. Graph condensation: A survey. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [16] Xinyi Gao, Wentao Zhang, Tong Chen, Junliang Yu, Hung Quoc Viet Nguyen, and Hongzhi Yin. 2023. Semantic-aware node synthesis for imbalanced heterogeneous information networks. In *Proceedings of the 32nd ACM international conference on information and knowledge management*. 545–555.
- [17] Xinyi Gao, Wentao Zhang, Junliang Yu, Yingxia Shao, Quoc Viet Hung Nguyen, Bin Cui, and Hongzhi Yin. 2024. Accelerating scalable graph neural network inference with node-adaptive propagation. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 3042–3055.
- [18] Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. 2017. Neural message passing for quantum chemistry. In *International Conference on Machine Learning*. PMLR, 1263–1272.
- [19] William L. Hamilton, Zitao Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4–9, 2017, Long Beach, CA, USA*. 1024–1034.
- [20] Ziniu Hu, Yuxiao Dong, Kuansan Wang, and Yizhou Sun. 2020. Heterogeneous graph transformer. In *Proceedings of the web conference 2020*. 2704–2710.
- [21] Zengfeng Huang, Shengzhong Zhang, Chong Xi, Tang Liu, and Min Zhou. 2021. Scaling up graph neural networks via graph coarsening. In *Proceedings of the 27th ACM SIGKDD conference on knowledge discovery & data mining*. 675–684.
- [22] Valter Hudovernik. 2024. Relational data generation with graph neural networks and latent diffusion models. In *NeurIPS 2024 Third Table Representation Learning Workshop*.
- [23] Nguyen Quoc Viet Hung, Huynh Huu Viet, Nguyen Thanh Tam, Matthias Weidlich, Hongzhi Yin, and Xiaofang Zhou. 2017. Computing crowd consensus with partial agreement. *IEEE Transactions on Knowledge and Data Engineering* 30, 1 (2017), 1–14.
- [24] Ihab F Ilyas, George Beskales, and Mohamed A Soliman. 2008. A survey of top-k query processing techniques in relational database systems. *ACM Computing Surveys (CSUR)* 40, 4 (2008), 1–58.
- [25] Nishtha Jatana, Sahil Puri, Mehak Ahuja, Ishita Kathuria, and Dishant Gosain. 2012. A survey and comparison of relational and non-relational database. *International Journal of Engineering Research & Technology* 1, 6 (2012), 1–5.
- [26] Martin Jurkovic, Valter Hudovernik, and Erik Štrumbelj. [n. d.]. SyntheRela: A Benchmark For Synthetic Relational Database Generation. In *Will Synthetic Data Finally Solve the Data Access Problem?*
- [27] Mohamed Amine Ketata, David Lüdke, Leo Schwinn, and Stephan Günnemann. 2025. Joint Relational Database Generation via Graph-Conditional Diffusion Models. *arXiv preprint arXiv:2505.16527* (2025).
- [28] Clement Lee and Darren J Wilkinson. 2019. A review of stochastic block models and extensions for graph clustering. *Applied Network Science* 4, 1 (2019), 1–50.
- [29] Ziming Li, Youhuan Li, Yuyu Luo, Guoliang Li, and Chuxu Zhang. 2025. Graph neural networks for databases: A survey. *arXiv preprint arXiv:2502.12908* (2025).
- [30] Yuxuan Liang, Wentao Zhang, Xinyi Gao, Ling Yang, Chong Chen, Hongzhi Yin, Yunhai Tong, and Bin Cui. 2025. Training-free heterogeneous graph condensation via data selection. In *2025 IEEE 41st International Conference on Data Engineering (ICDE)*. IEEE, 1718–1731.
- [31] Yunhui Liu, Xinyi Gao, Tiek He, Jianhua Zhao, and Hongzhi Yin. 2025. Teaching mlps to master heterogeneous graph-structured knowledge for efficient and accurate inference. *IEEE Transactions on Knowledge and Data Engineering* (2025).
- [32] Noel Loo, Ramin Hasani, Alexander Amini, and Daniela Rus. 2022. Efficient dataset distillation using random feature approximation. *Advances in Neural Information Processing Systems* 35 (2022), 13877–13891.
- [33] Timothy Nguyen, Zhourong Chen, and Jaehoon Lee. 2021. Dataset Meta-Learning from Kernel Ridge-Regression. In *ICLR*.
- [34] Neha Patki, Roy Wedge, and Kalyan Veeramachaneni. 2016. The Synthetic data vault. In *IEEE International Conference on Data Science and Advanced Analytics (DSAA)*. 399–410. doi:10.1109/DSAA.2016.49
- [35] Joshua Robinson, Rishabh Ranjan, Weihua Hu, Kexin Huang, Jiaqi Han, Alejandro Dobles, Matthias Fey, Jan Eric Lenssen, Yiwen Yuan, Zecheng Zhang, et al. 2024. Relbench: A benchmark for deep learning on relational databases. *Advances in Neural Information Processing Systems* 37 (2024), 21330–21341.
- [36] Michael Schlichtkrull, Thomas N Kipf, Peter Bloem, Rianne Van Den Berg, Ivan Titov, and Max Welling. 2018. Modeling relational data with graph convolutional networks. In *The Semantic Web: 15th International Conference, ESWC 2018, Heraklion, Crete, Greece, June 3–7, 2018, Proceedings 15*. Springer, 593–607.
- [37] Hung Vinh Tran, Tong Chen, Nguyen Quoc Viet Hung, Zi Huang, Lizhen Cui, and Hongzhi Yin. 2025. A thorough performance benchmarking on lightweight embedding-based recommender systems. *ACM Transactions on Information Systems* 43, 3 (2025), 1–32.
- [38] Hung Vinh Tran, Tong Chen, Guanhua Ye, Quoc Viet Hung Nguyen, Kai Zheng, and Hongzhi Yin. 2025. On-device content-based recommendation with single-shot embedding pruning: A cooperative game perspective. In *Proceedings of the ACM on Web Conference 2025*. 772–785.
- [39] Liane Vogel, Jan-Micha Bodensohn, and Carsten Binnig. 2024. Wikidbs: A large-scale corpus of relational databases from wikidata. *Advances in Neural Information Processing Systems* 37 (2024), 41186–41201.
- [40] Tongzhou Wang, Jun-Yan Zhu, Antonio Torralba, and Alexei A Efros. 2018. Dataset distillation. *ArXiv preprint* (2018).
- [41] Xiao Wang, Houye Ji, Chuan Shi, Bai Wang, Yanfang Ye, Peng Cui, and Philip S Yu. 2019. Heterogeneous graph attention network. In *The world wide web conference*. 2022–2032.
- [42] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E Sarma, Michael M Bronstein, and Justin M Solomon. 2019. Dynamic graph cnn for learning on point clouds. *ACM Transactions on Graphics (tog)* 38, 5 (2019), 1–12.
- [43] Zongwei Wang, Min Gao, Junliang Yu, Xinyi Gao, Quoc Viet Hung Nguyen, Shazia Sadiq, and Hongzhi Yin. 2025. Id-free not risk-free: Llm-powered agents unveil risks in id-free recommender systems. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1902–1911.
- [44] Zongwei Wang, Min Gao, Junliang Yu, Shazia Sadiq, Hongzhi Yin, and Ling Liu. 2025. When Graph Contrastive Learning Backfires: Spectral Vulnerability and Defense in Recommendation. *ACM Transactions on Information Systems* (2025).
- [45] Ziyu Wang, Yue Xu, Cewu Lu, and Yong-Lu Li. 2024. Dancing with still images: Video distillation via static-dynamic disentanglement. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 6296–6304.
- [46] Zongwei Wang, Junliang Yu, Min Gao, Hongzhi Yin, Bin Cui, and Shazia Sadiq. 2024. Unveiling vulnerabilities of contrastive recommender systems to poisoning attacks. In *Proceedings of the 30th ACM SIGKDD conference on knowledge discovery and data mining*. 3311–3322.
- [47] Max Welling. 2009. Herding dynamical weights to learn. In *Proceedings of the 26th Annual International Conference on Machine Learning*. 1121–1128.
- [48] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. How powerful are graph neural networks? *arXiv preprint arXiv:1810.00826* (2018).
- [49] Hongzhi Yin and Bin Cui. 2016. *Spatio-temporal recommendation in social media*. Springer.
- [50] Hongzhi Yin, Bin Cui, Ling Chen, Zhiting Hu, and Xiaofang Zhou. 2015. Dynamic user modeling in social media systems. *ACM Transactions on Information Systems*

- (TOIS) 33, 3 (2015), 1–44.
- [51] Hongzhi Yin, Liang Qu, Tong Chen, Wei Yuan, Ruiqi Zheng, Jing Long, Xin Xia, Yuhui Shi, and Chengqi Zhang. 2025. On-device recommender systems: A comprehensive survey. *Data Science and Engineering* (2025), 1–30.
 - [52] Ruonan Yu, Songhua Liu, and Xinchao Wang. 2023. Dataset distillation: A comprehensive review. *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2023).
 - [53] Han Zhang, Quan Gan, David Wipf, and Weinan Zhang. 2023. Gfs: Graph-based feature synthesis for prediction over relational databases. *arXiv preprint arXiv:2312.02037* (2023).
 - [54] Yanqiao Zhu, Weizhi Xu, Jinghao Zhang, Yuanqi Du, Jieyu Zhang, Qiang Liu, Carl Yang, and Shu Wu. 2021. A survey on graph structure learning: Progress and opportunities. *arXiv preprint arXiv:2103.03036* (2021).

A Appendix

A.1 Time Complexity

The overall pipeline of T2G comprises three main components: pretraining, graph structure generation, and feature distillation. We analyze the time complexity of each component separately. For the original relational database, let n_T , e_T , d_{num} , and d_{cat} denote the average number of entities, links, numerical columns, and categorical columns per table, respectively. The numbers of table and link types are represented by n and m . The average number of entities in the synthetic REG is denoted by n'_T , and the token and embedding dimensions are d_{token} and d .

- **Pretraining.** This stage includes column tokenization, K -layer graph convolution, and online clustering. The time complexities of these submodules are: $O(nn_T d_{token}(d_{num} + d_{cat}))$, $O(mKed)$, and $O(nn'_T n_T dt)$, where t denotes the number of iterations in the clustering.
- **Graph Structure Generation.** The generation process includes SBM estimation and adjacency sampling, which require $O(me_T)$ and $O(mn'^2_T)$, respectively.
- **Feature Distillation.** This stage involves K -step message passing in the HGNN and the closed-form solution of KRR. Their respective complexities are: $O(mKed)$ and $O(n'^3_T + n_T dD)$, where D denotes the dimension of the predictor head (i.e., the number of categories for classification or $D = 1$ for regression task).

In a nutshell, the time complexity of T2G is dominated by the pretraining and feature distillation stages. However, since $n'_T \ll n_T$ and efficient acceleration techniques (e.g., FAISS [5] and neighbor sampling in HGNNs) can be applied, the overall computational cost remains well controlled, ensuring scalability to large-scale RDBs.

A.2 Storage Efficiency Analysis (Q5)

Table 5 summarizes the storage comparison between the original databases and the synthetic data generated by T2G. Across all three benchmarks, T2G achieves remarkable data compression while preserving essential predictive information. For instance, Rossmann’s synthetic data retains only 0.02% of the original rows in the training set yet reduces the storage size from 35.0 MB to 61.1 KB, achieving over $500\times$ compression.

A small portion of the total storage comes from the tokenizers, which encode column-specific modality representations. Although tokenizers add only a few kilobytes overall, they are crucial for enabling consistent encoding of numerical and categorical attributes. Notably, Airbnb exhibits a relatively larger tokenizer size (23.0 KB) compared to Rossmann (5.0 KB) and Walmart (8.7 KB). This increase is attributed to Airbnb’s higher column dimensionality and richer categorical diversity, which require more embedding parameters to represent diverse attributes. Nevertheless, the tokenizer storage remains minimal compared to the original database size, contributing less than 0.01% of the total storage.

Table 5: Comparison of size and storage. r denotes the compression ratio, defined as the proportion of entities in the synthetic data relative to those in the original training set.

	Rossmann		Walmart		Airbnb	
	Original Database	Ours ($r=0.02\%$)	Original Database	Ours ($r=0.1\%$)	Original Database	Ours ($r=0.02\%$)
#Rows	1,015,159	147	310,707	192	1,020,723	119
Data	35.0M	59.1KB	13.5MB	80.8KB	350.0MB	46.8KB
Tokenizers	N/A	5.0KB	N/A	8.7KB	N/A	23.0KB
Total Storage	35.0M	61.1KB	13.5MB	89.5KB	350.0MB	69.8KB